

On the Formal Verification of Authenticated Encryption of the MQTT Protocol

Varsha Jarali^[0000–0003–3198–8448] and Shashi Kant Pandey^[0000–0002–0818–6984]

Society for Electronic Transactions and Security (SETS)

(Under O/o PSA to GOI)

Taramani, Chennai, India-600113

varshapjarali@gmail.com

shashikantshvet@gmail.com

Abstract. The Message Queuing Telemetry Transport (MQTT) protocol is highly preferable for Internet of Things (IoT) environments due to its lightweight architecture, but routing sensitive medical data through a central broker introduces severe privacy risks if the broker is untrusted or compromised. To address this, we propose secure MQTT, a high performance end to end encrypted (E2EE) protocol tailored for constrained devices that renders the broker completely blind to message payloads and incapable of man in the middle (MitM) attacks. Our design utilizes a nested AES-GCM encryption architecture that strictly separates link-level routing metadata from application layer confidentiality. To establish these secure channels efficiently, MQTT integrates MQTT v5.0 enhanced authentication key exchange mechanism via a challenge response embedding one time Broker Nonce into the Schnorr digital signatures version of HMQV key exchange protocol. This provide authenticated end to end session key derivation, that requires only a negligible computational increase over basic ECDH. The security of this proposed model has been rigorously proven using the ProVerif cryptographic verifier under the Dolev-Yao threat model, offering a highly secure, low overhead solution for modern IoT networks.

Keywords: Internet of Things · Formal verification · Cyber security · ProVerif · HMQV · MQTT.

Mathematics Subject Classification (MSC2020)

1 Introduction

The Internet of Things is very resource intensive, and its scope of use becomes wider day by day. The projection of the number of IoT products worldwide is 44 billion by 2025 [1]. An extensive work around the globe is happening for the safeguarding of IoT digital infrastructure. Cybersecurity Labelling for Internet of Things has been launched as the U.S. Cyber Trust Mark in May 2025 [2]. Singapore, Finland, Germany, and the Republic of Korea have launched a mutual recognition for the Cybersecurity Labelling Scheme for IoT - CLS(IoT) in the year 2021 [3].

The information security of an IoT device has three building blocks: a secure encryption scheme to protect data at all levels, rest, transit, and process; a robust authentication mechanism; and a foolproof protocol architecture. Security analysis is essential at every level of IoT devices. According to the 2015 Eclipse IoT survey, the most widely used messaging protocols are HTTP, MQTT, and CoAP. In the year 1999, IBM developed MQTT and made it open source in 2010. CoAP is another protocol, released in 2010.

Formal analysis is a prerequisite for higher assurance levels (Level 3–4) as per FIPS 140-3 [4] and Common Criteria (ISO/IEC 15408) [5] in high impact systems. Modern RFCs of the Internet Engineering Task Force (IETF) strongly expect security analysis for all crypto modules. The formal analysis requires a proof of functionally correct integration of all cryptographic components, such as:

- Information theoretically proved random number generators
- Theoretically proved secure crypto primitives
- Functionally correct protocols.

A source of randomness is essential for initiating secure communication sessions, as it plays a critical role in generating cryptographic parameters such as nonces, session keys, and challenge values. By integrating such randomness with proven and secure cryptographic modules, robust communication protocols can be established. Ensuring the correctness and validation of these protocols with respect to security requirements is a fundamental prerequisite. In this work, our primary focus is on the functional correctness and security analysis of communication protocols. Several formal verification tools and frameworks have been developed to analyze protocol functionality and security properties [7,8,9,10,11,12]. A comprehensive survey of current formalisms, tools, and approaches for analyzing the behavior of autonomous robotic systems is presented in [13]. Among the available verification tools, CryptoVerif and ProVerif are widely used open source frameworks that employ computational and symbolic models, respectively, for protocol verification. The scalability challenges and limitations associated with existing verification methods and tools have been extensively discussed in [14,15,16,17]. Formal verification using ProVerif has been applied in [6] to analyze the security of the EPS-AKA protocol designed for connectivity between 4G and 5G cellular networks. In addition, ensuring the functional correctness of application-layer protocols used in IoT systems is equally important. One such protocol is MQTT, an OASIS-standard lightweight messaging protocol [18]. The original MQTT protocol does not provide built in security mechanisms such as authentication and encryption. To address this limitation, the authors in [19] proposed an enhanced version of MQTT, namely PMQTT, which incorporates authentication and encryption capabilities for communicating entities. The Protected Message Queue Telemetry Transport (PMQTT) protocol is based on MQTT with added cryptographic primitives to offer security services for IoT systems. PMQTT involves three stages. In the first stage, authentication will be established between Patients and the Doctors through the server based on the elliptic curve digital signature algorithm (ECDSA). In the second stage, the

session key will be shared using the elliptic curve Diffie-Hellman (ECDH) key agreement protocol. And in the third stage, encrypting and decrypting the specific data will be exchanged using the shared secret key by the AES scheme. Furthermore, they formally verified the functional correctness of the proposed PMQTT protocol using ProVerif.

The Menezes–Qu–Vanstone (MQV) [20] protocol is a Diffie-Hellman scheme based authenticated key agreement protocol. In 1995, Scott Vanstone, Minghua Qu, and Alfred Menezes made the first proposal. In spite of the ambitious security goals providing by MQV, it turns out that MQV protocol falls short of delivering most of its intended security. The goals mentioned by authors of MQV were analyzed in well defined formal setting by [26], identifies MQV facing group representation attacks, UKS attacks, lack of PFS [25], KCI attacks and prime order checks. On the basis of these findings, Hugo [27] presented HMQV, a carefully designed variant of MQV, that provides the same performance and functionality of the original protocol.

Taking this information into account, we are proposing a secured MQTT protocol incorporating HMQV for authenticated key exchange and AES-GCM for authenticated encryption. The main contribution of this paper can be seen as the answer of following questions:

- ▷ How MQTT works with authenticated encryption?
- ▷ Propose a new protocol on top of MQTT with HMQV and AES
- ▷ Proverif modeling and formulation of Secure MQTT.

Organization of the paper: Section 1 introduces IoT protocols and their importance, along with an overview of the proposed protocol. Section 2 describes overview of Proverif along with detailed description of secrecy and authenticity. Section 3 details the three stage mechanism of proposed design. Section 4 presents the security claims of proposed design in Proverif for various attack scenarios. Finally, Section 5 concludes the article.

2 Overview of ProVerif

One of the assumptions for the strategy for verification relies on the methods to find the vulnerability in a system. Most protocol verifiers are based on a symbolic model, and with the perfect cryptographic assumption that the adversary can get the secret only if it has the decryption function with the correct key. The security testing is complete only if it has been done in the presence of a malicious adversary, and it is the foundation of Doleve and Yao model for automatic verification of protocols [21]. ProVerif [22] is a tool which verifies the functional correctness of protocols. The Horn clause resolution algorithm plays a major role at the core of this tool to derive security vulnerabilities. The security properties written in pi calculus, named queries, are the input for this protocol verifier tool. The tool takes queries to validate three security assumptions. The formulation of secrecy properties, authentication properties, and equivalence properties is done through proving the reachability property, correspondence assertion, and observational equivalence, respectively.

2.1 Secrecy and ProVerif

Proving reachability properties is one of the most fundamental capabilities of ProVerif. The tool enables the analysis of which terms are accessible to an attacker and, consequently, allows the evaluation of the (syntactic) secrecy of terms with respect to a given protocol model. To verify the secrecy of a term M , the query "query attacker(M)" must be included in the input file before the main process, where M is a ground term without destructors and may contain free names, including private names that are not initially known to the attacker. Internally, when ProVerif is provided with the query "query attacker(M).", it attempts to prove "not attacker(M)". Therefore, obtaining the result "RESULT not attacker(M)" is true. indicates that the protocol preserves the secrecy of M . Using such secrecy queries, the confidentiality of sensitive materials, including cryptographic keys, software updates, and other confidential data, can be formally verified.

2.2 Mutual Authentication and ProVerif

Authentication between communicating entities can be expressed through correspondence assertions in ProVerif. Authentication of A to B ensures that if B reaches the end of the protocol and believes that it has established a shared key k with A, then A was indeed the communicating party and has also shared the same key k . Similarly, authentication of B to A guarantees that if A completes the protocol with a shared key k , then B actually proposed k for communication with A. To verify such authentication properties, ProVerif uses correspondence assertions. In particular, injective correspondence assertions capture a one to one relationship between protocol events and are represented as: query $x_1 : t_1, \dots, x_n : t_n; inj - event(e(M_1, \dots, M_j)) \implies inj - event(e'(N_1, \dots, N_k))$.

Informally, this assertion states that for every occurrence of the event $e(M_1, \dots, M_j)$, there exists a distinct earlier occurrence of the event $e'(N_1, \dots, N_k)$. Consequently, the number of occurrences of $e'(N_1, \dots, N_k)$ is greater than or equal to the number of occurrences of $e(M_1, \dots, M_j)$. The injective property guarantees a one to one correspondence between protocol executions, thereby preventing replay attacks and ensuring stronger authentication guarantees. It is important to note that the use of inj-event before the implication operator $\implies$ does not affect the semantics of the query; injectivity only has significance when specified after the implication.

2.3 Forward secrecy and ProVerif

Long-term secrets and a session key to establish communication are two major components of a protocol. Mathematically, if sk is the session key and ltk is the long term key, then generally we can connect them with a secret function f by $sk = f(ltk)$. Here, f plays the role of a key derivation function (KDF). It must be a one way function to provide hardness for key related attacks on a protocol.

In the ProVerif manual, it was mentioned that Phase n is used to verify the forward secrecy. If $n = 1$ means in phase 1, all the processes that have not yet reached the phase $i \geq 1$ are discarded, and the process may then execute instructions under phase 1, but not under phase $i \geq 2$. The purpose of proving forward secrecy is to demonstrate that the secrets shared in sessions prior to the corruption are retained, even in the event that some participants are corrupted (i.e., their secret keys are disclosed to the attacker). The secret keys of the corrupted participants in phase 1 can be output in ProVerif to model corruption; the secrets of the sessions conducted in phase 0 should be kept safe.

3 Authenticated Encryption in a Communication Protocol

A communication protocols are the ultimate setting of connection data packets for two or more than two devices. Addition of any cryptographic layer on these protocols concern about the security and privacy of any user. Here we present the discussion for the authenticated encryption of HMQV and the MQTT protocol.

3.1 Authenticated Encryption in MQTT: AE-MQTT

Compared to the unauthenticated Diffie-Hellman key exchange, which is susceptible to MITM attacks, HMQV seeks to offer implicit authentication. This implies that, even in the key agreement phase itself, the parties can be sure they are speaking with the appropriate party specifically exchanging signatures and certificates. The integration of long term private keys into the shared secret computation provides this assurance. In the design of HMQV, both ephemeral (short term, session specific) and static (long term, authenticated) key pairs are necessary for the shared secret computation. This effectively "implicitly" verifies the legitimacy of the public keys used, since if an attacker attempts to tamper, the resulting shared secret will not match for the genuine parties. In other words it uses dual challenge response signatures(DCR) method, where the DH values exchanged by the parties act as challenges from which both parties compute a dual challenge response signatures. Because only the legitimate parties can compute this signature, it acts as an implicit proof of identity to verify with share digital certificate. The protocol can be adapted to work with elliptic curve cryptography (ECC), in which case it's known as ECHMQV. This is a common and efficient implementation.

For end to end data secrecy, the clients encrypt their payloads using the Advanced Encryption Standard (AES-GCM) symmetric cipher, keyed with the secret established via HMQV. Because the MQTT broker simply routes the traffic and does not have the shared session key, the data remains strictly confidential between the end users, even if the broker is compromised. By combining the standard HMQV protocol for end to end key exchange between clients, we can create a highly secure, highly efficient MQTT architecture. To accommodate the extra messages required by HMQV during the connection phase, this flow

utilizes the Enhanced Authentication feature introduced in MQTT v5.0, which allows a multi step challenge/response using AUTH packets. Figure 1 gives the brief flow of this architecture:

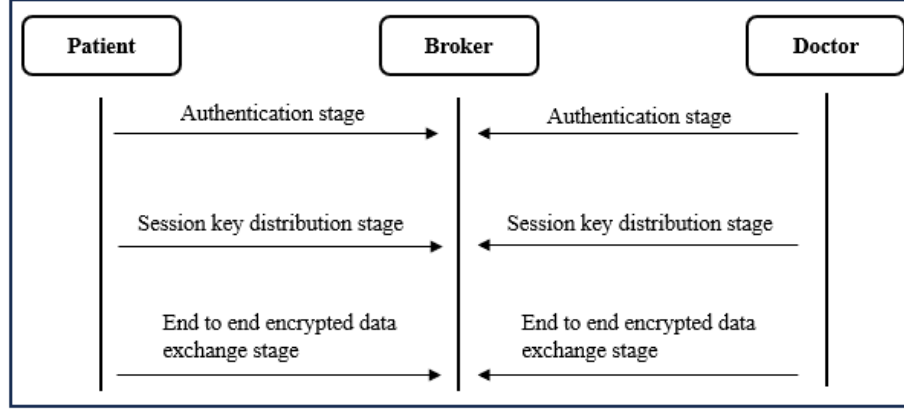


Fig. 1. Flow chart of the Protocol

Overview of HMQV-MQTT: The complete description of the processes executed by each party is presented below. Figure 2 illustrates the key derivation procedure and authenticated encryption mechanism among the participating entities. To explain the proposed protocol, a Patient–Doctor communication scenario from the healthcare domain is considered as an example. The protocol is described for communication between two parties; however, it can be naturally extended to support multiple communicating entities through a Broker. The proposed protocol follows the communication framework specified in MQTT v5.0 and is divided into three stages. In stage 1, authenticated key between the Clients and the Broker will be established. In stage 1, both the Patient and Doctor perform identical operations with the Broker to establish an authenticated key. Therefore, to maintain generality and avoid repetitive explanations, the term Client is used to represent both communicating entities. Stage 2 performs end to end key derivation between the communicating parties, and finally, in stage 3, secure encrypted data exchange takes place between the Doctor and the Patient through the Broker.

Stage 1: Clients-Broker Authentication (Using MQV): This stage guarantees that the Broker and the Clients explicitly confirm they both computed the exact same session key using Schnorr identification scheme of HMQV. This includes ephemeral key and signature generation initially to derive link-level key derivation (K_1 and K_2).

- Message 1 (CONNECT - Clients to Broker): The Patient (or Doctor) initiates the connection by sending an MQTT CONNECT packet to the Broker. The

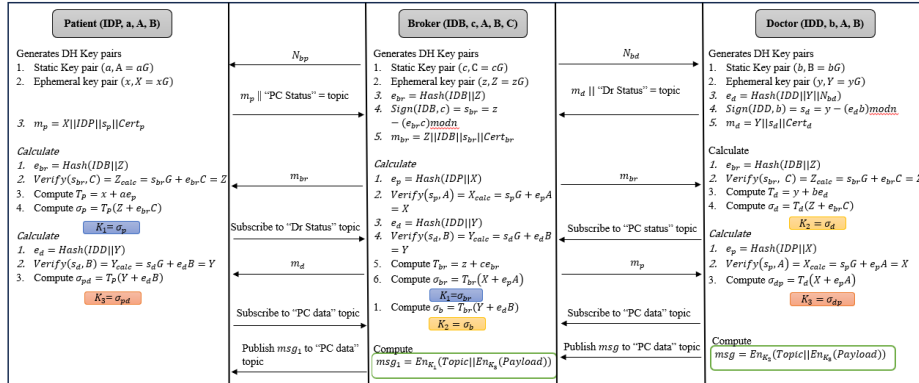


Fig. 2. Authenticated encryption of MQTT: Patient and broker, Doctor and Broker authenticate each other and establish shared secret to identify the topic via MQV. By using the shared secret established between Doctor and Patient, Doctor can send sensitive messages securely to the Patient without revealing the messages to the broker.

Broker generates a one time random number (the Nonce, N_{bp} or N_{bd}), and sends it to the Clients. Client generates his ephemeral Diffie-Hellman scalar x and public point ($X = xG$). Client mathematically binds its identity, the Broker's exact nonce, and X together into a hash (e_p). Client then signs this hash using his long term private key (a) to generate a Schnorr signature (s_p). Client (Patient) compute m_p and send it with a topic "PC status" to Broker. Similarly, Client (Doctor) compute m_d and send it with a topic "Dr status" to Broker.

- Message 3 (CONNACK - Broker to Clients): The Broker extracts the payload, verifies the digital certificate ($Cert_p$), and validates the signature against its original nonce. If valid, the Broker generates its own ephemeral key Z , computes the link-level symmetric key (K_1 for the Patient, K_2 for the Doctor), and sends an MQTT CONNACK packet. The Broker attaches its own status (Z , s_{br} , $Cert_{br}$) in the final Authentication Data field.
- Message 4 (AUTH - Clients and Broker): The client receives m_{br} , extracts the payload, verifies the digital certificate ($Cert_p$), and validates the signature. Computes the link-level symmetric keys (K_1 for the Patient, K_2 for the Doctor).

Stage 2: End to End Key Derivation (K_3): Now, the Patient-Doctor devices are securely connected to the Broker via link keys (K_1 , K_2), they must exchange their ephemeral keys with each other to derive the End to End key (K_3). They do this using standard MQTT Pub/Sub mechanics on designated "Status Topics".

- The Patient sends a SUBSCRIBE packet to the topic "Dr Status", conversely, the Doctor sends a SUBSCRIBE packet to "PC Status" to Broker. The Broker replies with a SUBACK packet m_d to Patient and m_p to Doctor

- Publishing Ephemeral Keys: The Doctor receives the Patient's PUBLISH packet. The Doctor natively verifies the Patient's signature and certificate (completely bypassing the Broker's trust layer). Similarly, Patient receives the Doctor's PUBLISH packet. Then Patient natively verifies the Doctor's signature and certificate.
- Computing the End to End Key: Both parties then compute the tripartite HMQV algebraic equation using their own private keys and the received public keys to derive the identical shared secret (K_3).

Stage 3: End to End Encrypted Data Exchange: Using AES-GCM with securely established K_3 , the devices can now stream sensitive medical data without the Broker being able to read or alter it. This utilizes designated "Data Topics".

- Topic Subscriptions: Both Patient and Doctor subscribe to the PC Data topic on the Broker.
- Encryption: The Doctor encrypts the medical payload using AES-CCM with the secret key K_3 . To ensure the Broker cannot read routing metadata, the Doctor wraps this encrypted payload and the target Topic into a second AES-CCM layer using the link-level key K_2 .
- Publishing Data: Doctor publishes the ciphertext to the "PC Data" topic. The Broker decrypt using link-level key K_2 . Verify the topic, wraps the still encrypted inner payload using the Patient's link-level key K_1 and forwards the PUBLISH packet to the Patient.
- Decryption: Patient receives the PUBLISH packet on the "PC Data" topic. It decrypts the outer layer using its link-level key K_1 , and then decrypts the inner payload using the key K_3 to access the plaintext medical data.

By using this flow, the network achieves explicitly confirmed access control at the Broker level (via HMQV) and strict, implicit end to end data confidentiality between clients (via standard HMQV and AES-GCM), all while remaining significantly lighter computationally.

4 Security Analysis

The proposed design is formally verified in Proverif. For the complete analysis, queries have been asked for end to end payload secrecy, link-level replay resistance, profound resilience against both key compromise impersonation (KCI) and unknown key share (UKS) attacks, and mutual authenticity.

4.1 Formal proof for AE-MQTT

As verified mathematically in ProVerif, proposed protocol successfully defeats the following advanced attacks:

1. Eavesdropping and Man in the Middle (MITM) Attacks: The protocol utilizes nested AES-CCM encryption. The outer link keys (K_1, K_2) prevent outside observers from reading metadata. More importantly, the inner end to end key (K_3) prevents the Broker (the MITM) from reading or altering the medical payload.
2. Known Key Attacks (KKA): If an attacker compromises a past session key (e.g., by hacking the memory of a device during a specific session), it does not help them decrypt future sessions. This is because every session uses freshly generated, random ephemeral keys (X, Y) to calculate the new session key.
3. Proof of Secrecy, Confidentiality, Eavesdropping, MITM, and Known Key Attacks (KKA) : To prove that the end to end data remains perfectly confidential against eavesdroppers, MITM attackers, and attackers who compromise past session keys (KKA), we define the payload as a private free name and ask ProVerif if the Dolev-Yao attacker can ever compute it.
4. Mutual Authenticity: Instead of sending an explicit digital signature MQV authenticates the communicating parties implicitly Schnorr variant of it. It is proven that HMQV can be transformed via Fiat Shamir methodology into a signature scheme, that is provably secure in random oracle model. Moreover, Fiat Shamir transformation based signature scheme are quantum safe. To prove that the payload was genuinely sent by the honest peer without tampering or impersonation, we use a strict injective correspondence query. This ensures a one to one mapping: every single time the Patient successfully receives and decrypts a payload, the Doctor distinctly sent it.
5. Key Compromise Impersonation (KCI): If an attacker compromises an IoT device's long term private key, they can impersonate that device. However, HMQV guarantees they cannot use that compromised key to reverse the attack and impersonate other, uncorrupted devices to the victim. This is an inherent mathematical strength of the HMQV derivation. If an attacker steals the Patient's long term private key, they cannot use it to spoof messages pretending to be the Doctor. To successfully forge the K_3 derivation as the Doctor, the attacker must explicitly possess the Doctor's private key b . To prove this, we explicitly give the attacker the ability to leak private keys via an event ($Leak_{Priv}$). The query proves that the Patient will only derive the end to end key K_3 if the honest Doctor actually participated, or if the attacker explicitly stole the Doctor's private key.
6. Unknown Key Share (UKS) Attacks: In a UKS attack, an adversary tricks two parties into establishing a secure key while confusing them about who is on the other end. Protocol blocks this because the identities (IDP, IDD) are explicitly hashed into the ephemeral bindings (e_p, e_d). It is mathematically impossible for the Patient and Doctor to compute the same K_3 without agreeing perfectly on each other's identities.
7. Link-Level Replay Attacks: The inclusion of the Broker's one time challenge nonce (N_{bp} and N_{bd}) strictly blocks replays. If an attacker intercepts a valid handshake and replays it 100 times, the Broker will drop the connection because the signature is mathematically bound to previous expired nonce.

The verification proverif summary of a given design is given in the following diagram.

```

Verification summary:
Query not attacker_pl(Payload[]) cannot be proved.
Query inj-event(patient_receives_payload(idd, idp, p)) ==> inj-event(doctor_sends_payload(idd, idp, p)) cannot be proved.
Query inj-event(broker_computes_k1(idp, idb, x_1, k)) ==> inj-event(patient_sends_request(idp, idb, x_1)) is false.
Query inj-event(broker_computes_k2(idd, idb, y_1, k)) ==> inj-event(doctor_sends_request(idd, idb, y_1)) cannot be proved.
Query event(patient_computes_k3(idp, idd, idb, x_1, y_1, k3)) ==> event(doctor_sends_request(idd, idb, y_1)) is false.
Query event(doctor_computes_k3(idd, idp, idb, x_1, y_1, k3)) ==> event(patient_sends_request(idp, idb, x_1)) is false.
Query inj-event(patient_computes_k3(idp, idd, idb, x_1, y_1, k3)) ==> inj-event(doctor_sends_request(idd, idb, y_1)) is false.
Query inj-event(doctor_computes_k3(idd, idp, idb, x_1, y_1, k3)) ==> inj-event(patient_sends_request(idp, idb, x_1)) is false.
Query event(patient_computes_k3(idp, idd, idb, x_1, y_1, k3)) ==> event(doctor_sends_request(idd, idb, y_1)) || event(Leak_Priv(idd)) is true.
Query event(doctor_computes_k3(idd, idp, idb, x_1, y_1, k3)) ==> event(patient_sends_request(idp, idb, x_1)) || event(Leak_Priv(idp)) is true.
Query event(patient_computes_k3(idp, idd, idb, x_1, y_1, k3)) && event(doctor_computes_k3(idd', idp', idb', x', y', k3)) ==> idp = idp' && idd = idd' is true.

```

Fig. 3. Verification summary of all queries mentioned above

5 Conclusion

The proposed secure MQTT protocol provides a high performance, end to end encrypted (E2EE) architecture tailored for constrained IoT environments by using HMQV and nested AES-CCM encryption to strictly separate link-level routing metadata from application layer data confidentiality. At its core, the design integrates MQTT v5.0 enhanced authentication via a challenge response mechanism embedding one time Broker Nonces into Schnorr digital signatures to defeat link-level replay attacks alongside a highly efficient 2-message tripartite HMQV style key exchange for secure session key derivation. Rigorous formal verification using the ProVerif automated cryptographic verifier under the Dolev-Yao threat model mathematically proves that this architecture is exceptionally secure; it guarantees payload secrecy, mutual authentication, and profound resilience against man in the middle (MitM), key compromise impersonation (KCI), and unknown key share (UKS) attacks. Future work will focus on analyzing the protocol's computational performance within a practical, real world MQTT network, exploring the integration of synchronized redundant backup brokers to eliminate the central broker as a single point of failure, and investigating continuous ratcheting mechanisms for environments requiring strict historical data protection.

References

1. Ministry of Internal Affairs and Communications. (n.d.). Japan: IoT product security conformity assessment scheme policy draft. Government of Japan. <https://www.soumu.go.jp/main-content/000734465.pdf>

2. The White House. (2025, January 7). <https://bidenwhitehouse.archives.gov/briefing-room/statements-releases/2025/01/07/white-house-launches-u-s-cyber-trust-mark-providing-american-consumers-an-easy-label-to-see-if-connected-devices-are-cybersecure/>
3. Cyber Security Agency of Singapore. (2020). About Cybersecurity Labelling Scheme for IoT – CLS(IoT). <https://www.csa.gov.sg/Programmes/cybersecurity-labelling-scheme>
4. National Institute of Standards and Technology, Security Requirements for Cryptographic Modules, FIPS PUB 140-3, Mar. 2019.
5. International Organization for Standardization and International Electrotechnical Commission, Information security, cybersecurity and privacy protection — Evaluation criteria for IT security, ISO/IEC 15408 (Parts 1–3), 2022.
6. Lai, C., Li, H., Lu, R., Shen, X. (2013). SE-AKA: A secure and efficient group authentication and key agreement protocol for LTE networks. *Computer Networks*, 57(17), 3492–3510. <https://doi.org/10.1016/j.comnet.2013.08.001>
7. Blanchet B, Smyth B, Cheval V, Sylvestre M. ProVerif 2.00: Automatic Cryptographic Protocol Verifier, User Manual and Tutorial.
8. Project leader, Bruno Blanchet, Aymeric Fromherz, Charlie Jacomme, Benjamin Lipp CryptoVerif 2.00: Automatic Cryptographic Protocol Verifier, User Manual and Tutorial.
9. Nipkow, T., Paulson, L. C., Wenzel, M. (2018). Isabelle/HOL—A Proof Assistant for Higher-Order Logic. Technische Universität München. Retrieved from <https://isabelle.in.tum.de/doc/tutorial.pdf>
10. <https://tamarin-prover.com/>
11. Thierry Coquand, Gérard Huet, Christine Paulin-Mohring, Bruno Barras, Jean-Christophe Filliâtre, Hugo Herbelin, Chetan Murthy, Yves Bertot, Pierre Castéran, Rocq Prover, <https://rocq-prover.org/>
12. Mizar Proof Checker. Available at <http://mizar.org/>
13. Luckcuck, M., Farrell, M., Dennis, L., Dixon, C., and Fisher, M. (2019). Formal specification and verification of autonomous robotic systems: A survey. *ACM Computing Surveys*, 52(5), Article 100.
14. R. Kneuper, “Limits of Formal Methods,” *Formal Aspects of Computing*, vol. 9, no. 4, pp. 379–394, 1997. [Online]. Available: <https://www.kneuper.de/English/Publications/limits-formal-methods.pdf>
15. J. A. Hall, “Seven Myths of Formal Methods,” *IEEE Software*, vol. 7, no. 5, pp. 11–19, Sept. 1990.
16. G. J. Holzmann, “Design and Validation of Computer Programs,” in *Proc. IEEE Symp. Comput. Commun.*, pp. 143–152, 1988.
17. M. H. ter Beek, R. Chapman, R. Cleaveland, H. Garavel, R. Gu, I. ter Horst, J. J. A. Keiren, T. Lecomte, M. Leuschel, K. Y. Rozier, A. Sampaio, C. Seceleanu, M. Thomas, T. A. C. Willemse, and L. Zhang, “Formal Methods in Industry,” *Formal Aspects of Computing*, vol. 37, no. 1, Art. 7, pp. 1–38, Dec. 2024.
18. OASIS, “MQTT Version 5.0,” OASIS Standard, Mar. 2019
19. Elemam, E., Bahaa-Eldin, A. M., Shaker, N. H., Sobh, M. (Year). Formal verification for a PMQTT protocol. *Egyptian Informatics Journal*. <https://doi.org/DOI-HERE>
20. A. Menezes, M. Qu, and S. Vanstone, “Some new key agreement protocols providing mutual implicit authentication”, *Second Workshop on Selected Areas in Cryptography (SAC 95)*, pp. 22–32, 1995.

21. Danny Dolev and Andrew C. Yao (1983): On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* IT-29(12), pp. 198–208, doi:10.1109/TIT.1983.10566650.
22. Bruno Blanchet. The Security Protocol Verifier ProVerif and its Horn Clause Resolution Algorithm. *Electronic Proceedings in Theoretical Computer Science*, 2022, 373, pp.14– 22. 10.4204/eptcs.373.2hal-03897677
23. E. Barker, Q. Dang, S. Frankel, K. Scarfone, and P. Wouters, NIST Special Publication 800-77 Revision 1: Guide to IPsec VPNs. Gaithersburg, MD, USA: National Institute of Standards and Technology, Apr. 2023. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-77r1>
24. E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, Aug. 2018. [Online]. Available: <https://doi.org/10.17487/RFC8446>
25. Krawczyk, H. (2025). Perfect Forward Secrecy. In: Jajodia, S., Samarati, P., Yung, M. (eds) *Encyclopedia of Cryptography, Security and Privacy*. Springer, Cham. <https://doi.org/10.1007/978-3-030-71522-9-90>
26. Canetti, R., and Krawczyk, H., “Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels”, *Eurocrypt’2001*, LNCS Vol. 2045. Full version in: <http://eprint.iacr.org/2001/040>.
27. Krawczyk, Hugo, HMQV: a high-performance secure diffie-hellman protocol, *Proceedings of the 25th Annual International Conference on Advances in Cryptology, CRYPTO’05*, Springer-Verlag, 21, 546–566, 2005, doi/10.1007/1153521833